

Informe de Práctica --- ASP.NET Core MVC

Proyecto: MiPrimerMVC --- Rama: feature/practica-aspnet

Nombre: Bryam Romero Mendez

Curso/Paralelo: Software "5" A

Repositorio GitHub del proyecto: <https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC..git>

Introducción

Este informe documenta la extensión del proyecto ASP.NET Core MVC "MiPrimerMVC", desarrollado como práctica del curso. Se detallan los cinco requisitos solicitados: tabla HTML de productos, acción de eliminación con confirmación y protección CSRF, validación en cliente y servidor, publicación en la rama feature/practica-aspnet, y documentación README. Cada sección incluye el código relevante y un espacio reservado para la captura de pantalla correspondiente.

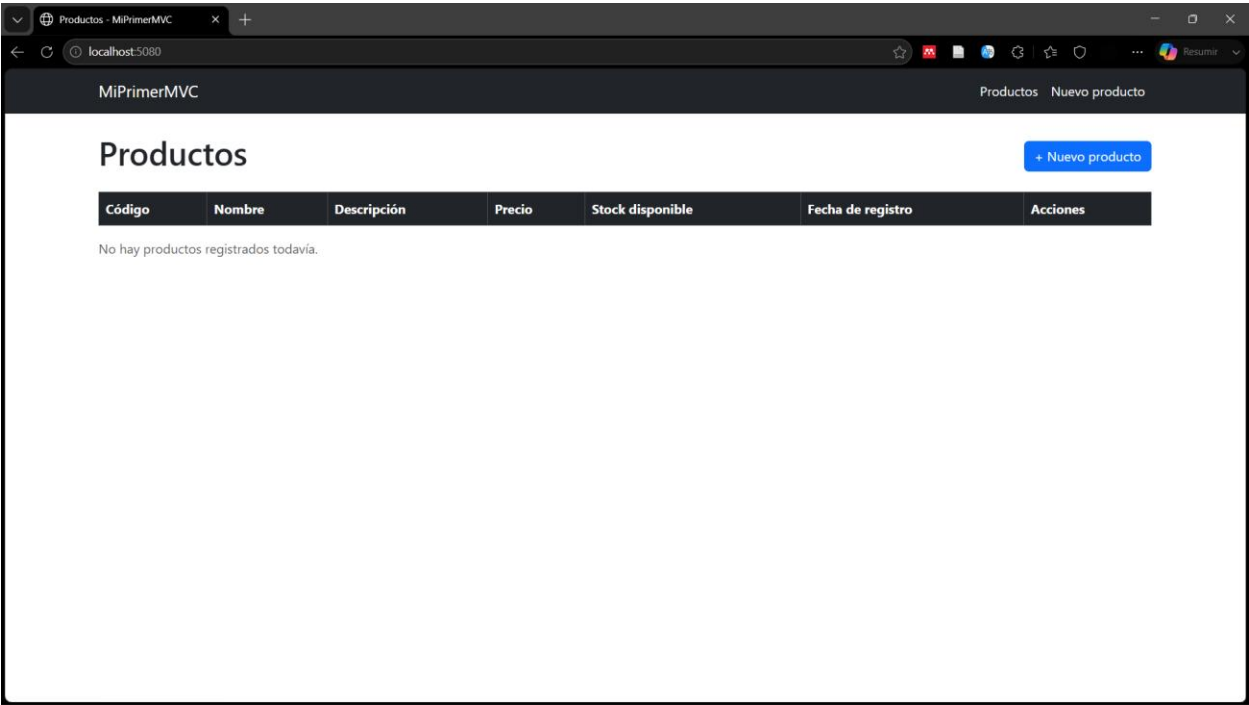
1. Tabla HTML con lista de productos

La vista Index.cshtml del controlador ProductoController renderiza una tabla HTML con todos los productos registrados, usando Tag Helpers de Razor (Html.DisplayNameFor / Html.DisplayFor) para mantener sincronizados los encabezados con las propiedades del modelo Producto.

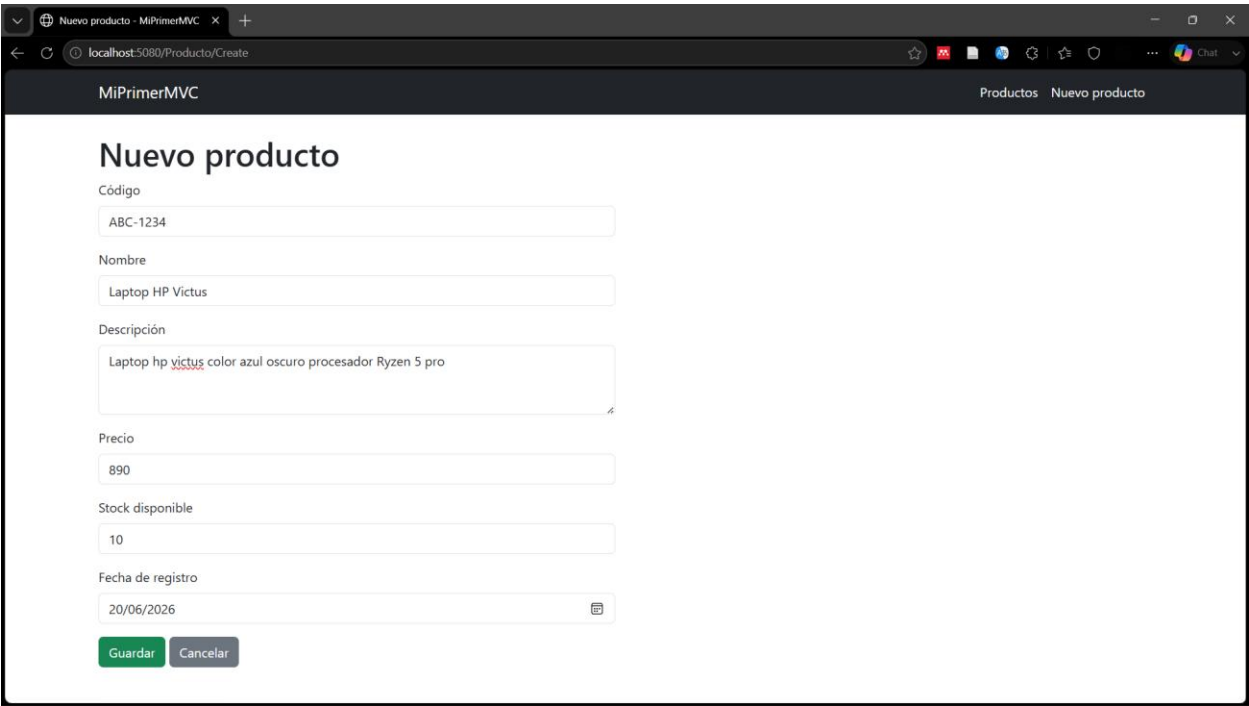
Código — Views/Producto/Index.cshtml (fragmento)

```
<table class="table table-striped table-bordered align-middle">
  <thead class="table-dark">
    <tr>
      <th>@Html.DisplayNameFor(model => model.Codigo)</th>
      <th>@Html.DisplayNameFor(model => model.Nombre)</th>
      <th>@Html.DisplayNameFor(model => model.Precio)</th>
      <th>@Html.DisplayNameFor(model => model.Stock)</th>
      <th>Acciones</th>
    </tr>
  </thead>
  <tbody>
    @foreach (var item in Model)
    {
      <tr>
        <td>@Html.DisplayFor(modelItem => item.Codigo)</td>
        <td>@Html.DisplayFor(modelItem => item.Nombre)</td>
        <td>@item.Precio.ToString("C")</td>
        <td>@Html.DisplayFor(modelItem => item.Stock)</td>
        <td>
          <a asp-action="Delete" asp-route-id="@item.Id"
            class="btn btn-sm btn-outline-danger">Eliminar</a>
        </td>
      </tr>
    }
  </tbody>
</table>
```

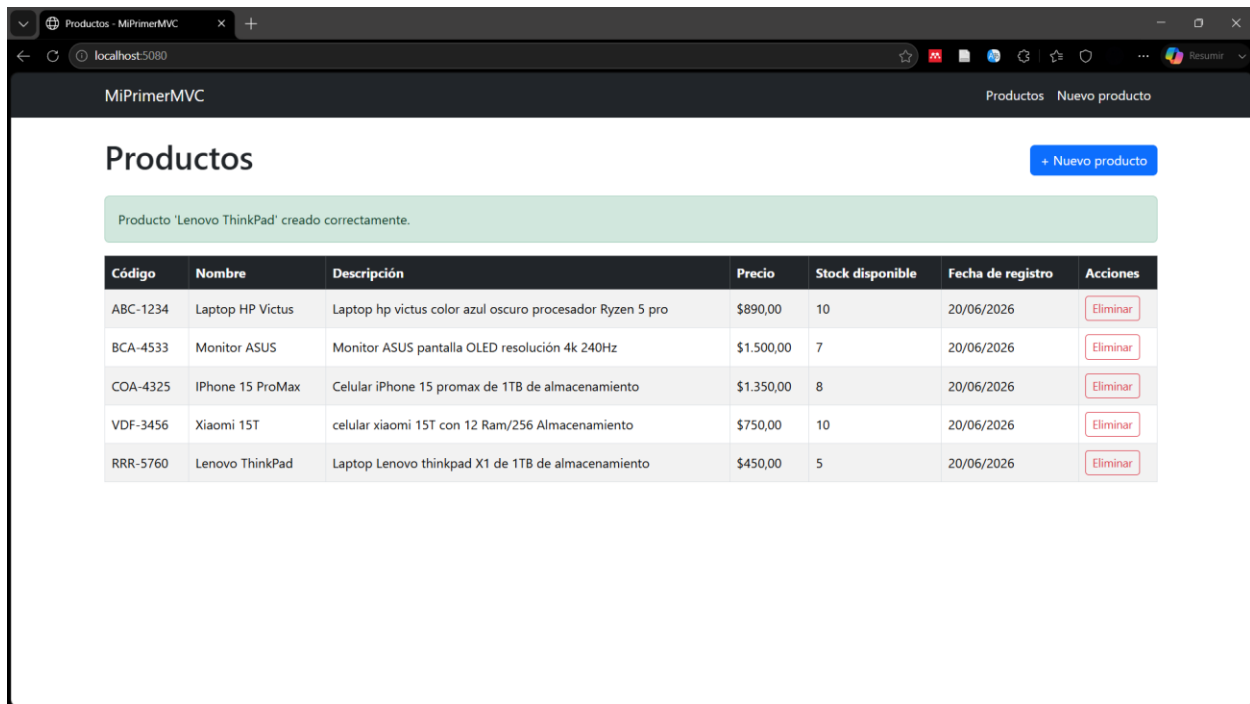
Lista vacía sin productos



Registro de nuevos productos



Lista con productos agregados



2. Acción Delete con confirmación POST y CSRF

Se implementó el patrón estándar de dos pasos: un GET que muestra una pantalla de confirmación (sin alterar datos) y un POST, protegido con [ValidateAntiForgeryToken], que ejecuta el borrado real. Esto evita que un simple enlace, prefetch del navegador o un robot de indexación pueda eliminar un registro accidentalmente.

Código — Controllers/ProductoController.cs (fragmento)

```
// GET: /Producto/Delete/5 – solo muestra confirmación
public IActionResult Delete(int? id)
{
    if (id == null) return NotFound();

    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto == null) return NotFound();

    return View(producto);
}

// POST: /Producto/Delete/5 – aquí se ejecuta el borrado real
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto == null)
    {
        TempData["MensajeError"] = "El producto ya no existe.";
        return RedirectToAction(nameof(Index));
    }

    _productos.Remove(producto);
    TempData["Mensaje"] = $"Producto '{producto.Nombre}' eliminado.";
    return RedirectToAction(nameof(Index));
}
```

Código — Views/Producto/Delete.cshtml (fragmento)

```
<form asp-action="Delete" method="post">
  <input type="hidden" asp-for="Id" />
  @Html.AntiForgeryToken()
  <button type="submit" class="btn btn-danger">Sí, eliminar</button>
  <a asp-action="Index" class="btn btn-secondary">Cancelar</a>
</form>
```

El inspector de código muestra las peticiones y los métodos GET antes de eliminar un producto

The screenshot shows a web browser at localhost:5080/Productos/Delete/3. The page title is 'MiPrimerMVC' and the breadcrumb is 'Productos > Nuevo producto'. The main heading is 'Eliminar producto'. A yellow warning box says: '¿Está seguro de que desea eliminar este producto? Esta acción no se puede deshacer.' Below this is a box containing product details: Código: COA-4325, Nombre: iPhone 15 ProMax, Descripción: Celular iPhone 15 promax de 1TB de almacenamiento, Precio: \$1.350,00, Stock disponible: 8. At the bottom are two buttons: 'Sí, eliminar' (red) and 'Cancelar' (gray). The Chrome DevTools Network tab is open, showing a list of requests. The first request is a GET to localhost, which is highlighted. The status is 200, and the type is document. The size is 3.8 kB and the time is 133 ms.

Después de dar clic en *eliminar producto* el método cambia a POST

The screenshot shows the 'Productos' page after clicking 'Sí, eliminar'. A green success message says: 'Producto 'iPhone 15 ProMax' eliminado correctamente.' Below this is a table with 7 columns: Código, Nombre, Descripción, Precio, Stock disponible, Fecha de registro, and Acciones. The table contains 4 rows of product data. The 'Acciones' column has an 'Eliminar' button for each row. The Chrome DevTools Network tab is open, showing a list of requests. The first request is a POST to localhost, which is highlighted. The status is 302, and the type is document. The size is 412 B and the time is 74 ms. A red dashed box highlights the first row of the table and the first request in the Network tab.

Código	Nombre	Descripción	Precio	Stock disponible	Fecha de registro	Acciones
ABC-1234	Laptop HP Victus	Laptop hp victus color azul oscuro procesador Ryzen 5 pro	\$890,00	10	20/06/2026	Eliminar
BCA-4533	Monitor ASUS	Monitor ASUS pantalla OLED resolución 4k 240Hz	\$1.500,00	7	20/06/2026	Eliminar
VDF-3456	Xiaomi 15T	celular xiaomi 15T con 12 Ram/256 Almacenamiento	\$750,00	10	20/06/2026	Eliminar
RRR-5760	Lenovo ThinkPad	Laptop Lenovo thinkpad X1 de 1TB de almacenamiento	\$450,00	5	20/06/2026	Eliminar

Productos - MiPrimerMVC

localhost:5080

MiPrimerMVC

Productos Nuevo producto

Productos

+ Nuevo producto

Producto 'iPhone 15 ProMax' eliminado correctamente.

Código	Nombre	Descripción	Precio	Stock disponible	Fecha de registro	Acciones
ABC-1234	Laptop HP Victus	Laptop hp victus color azul oscuro procesador Ryzen 5 pro	\$890,00	10	20/06/2026	Eliminar
BCA-4533	Monitor ASUS	Monitor ASUS pantalla OLED resolución 4k 240Hz	\$1.500,00	7	20/06/2026	Eliminar
VDF-3456	Xiaomi 15T	celular xiaomi 15T con 12 Ram/256 Almacenamiento	\$750,00	10	20/06/2026	Eliminar
RRR-5760	Lenovo ThinkPad	Laptop Lenovo thinkpad X1 de 1TB de almacenamiento	\$450,00	5	20/06/2026	Eliminar

Network

Filter

Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Waasm Other

Name

3

localhost

localhost

content.css

content.js

bootstrap.bundle.min.js

bootstrap.min.css

content.css

content.js

bootstrap.bundle.min.js

bootstrap.min.css

content.js

bootstrap.bundle.min.js

bootstrap.min.css

3

Request URL

http://localhost:5080/Producto/Delete/3

Request Method

POST

Status Code

302 Found

Remote Address

[::1]:5080

Referer Policy

strict-origin-when-cross-origin

Response headers

Raw

Content-Length

0

Date

Sat, 20 Jun 2026 20:01:51 GMT

Location

/

Server

Kestrel

Set-Cookie

ASP.NETCore.Mvc.CookieTempDataProvider=CID8lqbughI095KwoGhwCwOyIRMG5UEdPWWjPIRuQvbaum_d1ThQseldSk8DV8nkp5XM2Jr08-NEDW-3gqZZW_X0mOvYha7E2y5t5pR9N9w9wOfmDhAuzP-seOrv9QFMds00NFVNdSF69q3PS6wG-rV68s_EduHDe3yLHj0ZUSex15ZG8M4M1CqjE66SUBGQM; path=/; samesite=loahttponly

16 requests · 2.7 MB transferred · 3 · Request Headers

Productos - MiPrimerMVC

localhost:5080

MiPrimerMVC

Productos Nuevo producto

Productos

+ Nuevo producto

Producto 'iPhone 15 ProMax' eliminado correctamente.

Código	Nombre	Descripción	Precio	Stock disponible	Fecha de registro	Acciones
ABC-1234	Laptop HP Victus	Laptop hp victus color azul oscuro procesador Ryzen 5 pro	\$890,00	10	20/06/2026	Eliminar
BCA-4533	Monitor ASUS	Monitor ASUS pantalla OLED resolución 4k 240Hz	\$1.500,00	7	20/06/2026	Eliminar
VDF-3456	Xiaomi 15T	celular xiaomi 15T con 12 Ram/256 Almacenamiento	\$750,00	10	20/06/2026	Eliminar
RRR-5760	Lenovo ThinkPad	Laptop Lenovo thinkpad X1 de 1TB de almacenamiento	\$450,00	5	20/06/2026	Eliminar

3. Validación en cliente y servidor

El modelo Producto utiliza Data Annotations (Required, StringLength, Range, RegularExpression) que generan automáticamente validación en dos capas: en el cliente, mediante jQuery Validation Unobtrusive (feedback inmediato sin recargar la página); y en el servidor, mediante ModelState.IsValid dentro de la acción POST, que actúa como última línea de defensa incluso si JavaScript está deshabilitado.

Código — Models/Producto.cs (fragmento)

```
[Required(ErrorMessage = "El nombre del producto es obligatorio.")]
[StringLength(100, MinimumLength = 3,
    ErrorMessage = "El nombre debe tener entre 3 y 100 caracteres.")]
public string Nombre { get; set; } = string.Empty;

[Required(ErrorMessage = "El código del producto es obligatorio.")]
[RegularExpression(@"^[A-Z]{3}-\d{4}$",
    ErrorMessage = "El código debe tener el formato ABC-1234.")]
public string Codigo { get; set; } = string.Empty;

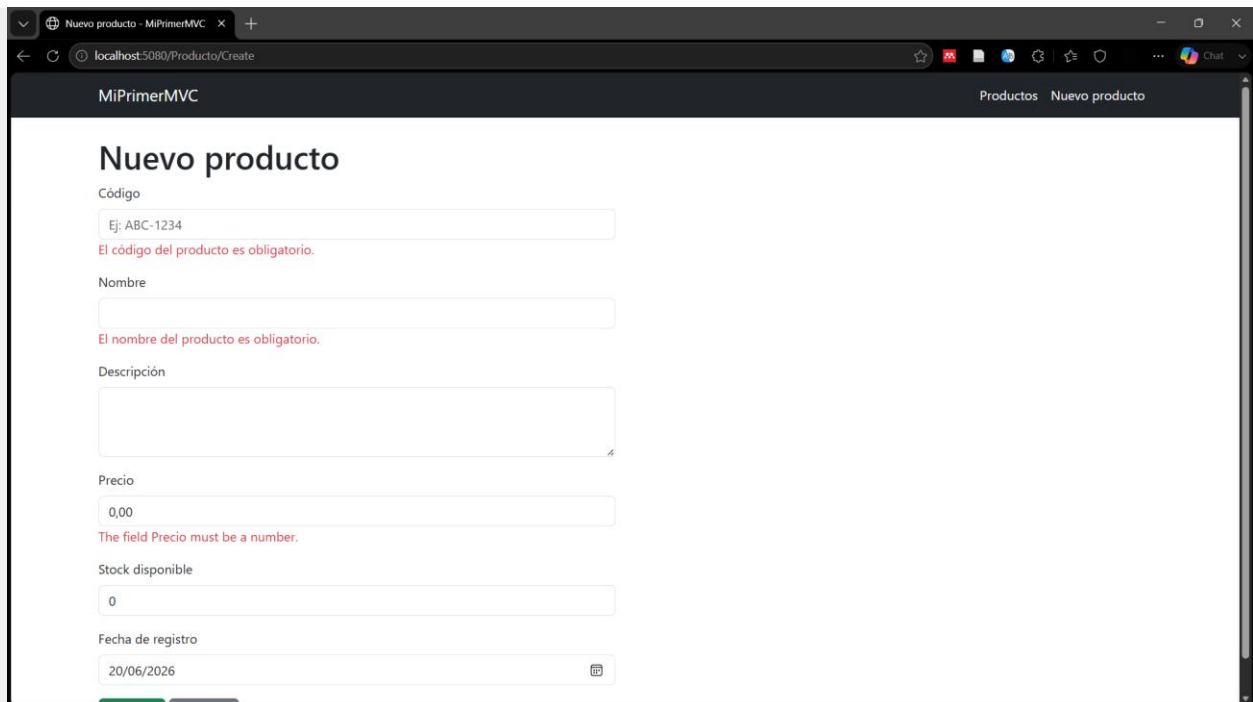
[Required(ErrorMessage = "El precio es obligatorio.")]
[Range(0.01, 1000000, ErrorMessage = "El precio debe estar entre 0.01 y 1,000,000.")]
public decimal Precio { get; set; }
```

Código — Controllers/ProductoController.cs (validación server-side)

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(Producto producto)
{
    if (_productos.Any(p => p.Codigo == producto.Codigo))
    {
        ModelState.AddModelError(nameof(producto.Codigo),
            "Ya existe un producto con este código.");
    }

    if (!ModelState.IsValid)
    {
        return View(producto); // se reenvían los errores a la vista
    }
    // ... continúa el guardado
}
```

Validación en el cliente (sin envío del formulario)



Nuevo producto - MiPrimerMVC

localhost:5080/Producto/Create

MiPrimerMVC

Productos Nuevo producto

Nuevo producto

Código

Ej: ABC-1234

El código del producto es obligatorio.

Nombre

El nombre del producto es obligatorio.

Descripción

Precio

0,00

The field Precio must be a number.

Stock disponible

0

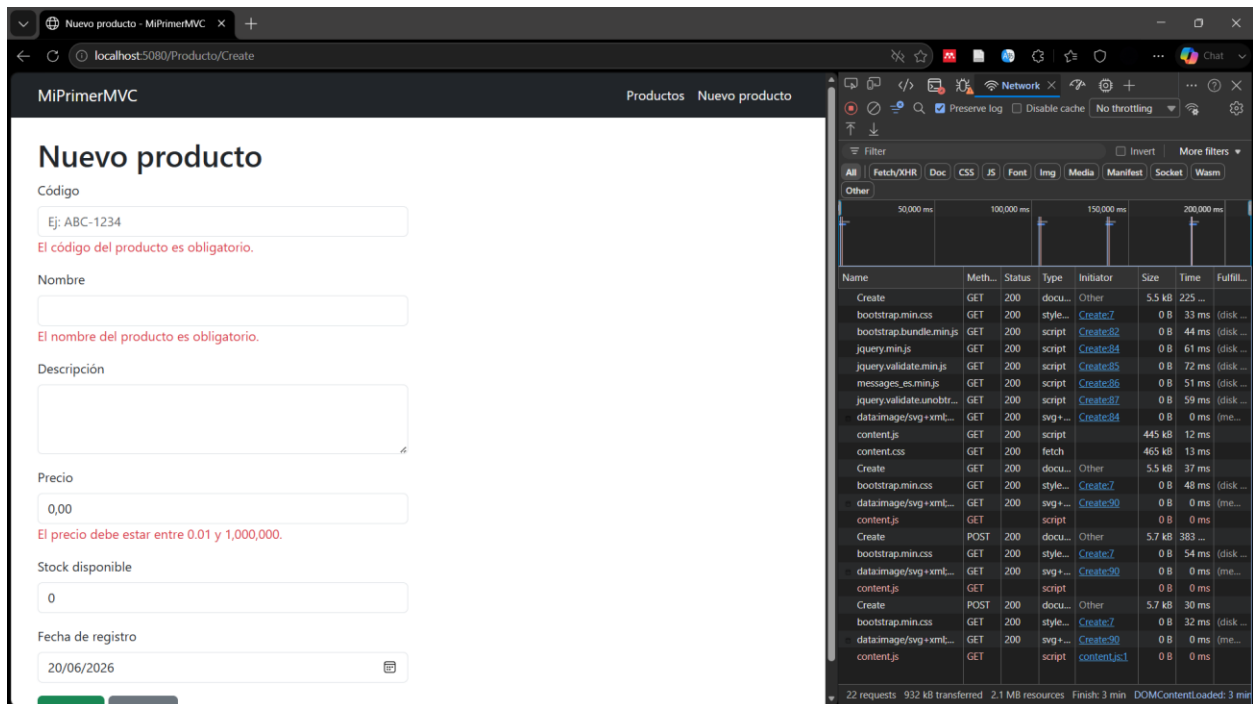
Fecha de registro

20/06/2026

Esta imagen es una captura del formulario para crear un producto (/Producto/Create) justo cuando el usuario deja un campo inválido (evento blur/focusout) sin haber pulsado el botón "Guardar". El error se muestra inmediatamente, mediante JavaScript en el navegador (jQuery Validation + jQuery Validation Unobtrusive), que interpreta los atributos data-val-* producidos automáticamente por las Data Annotations del modelo Producto (como data-val-required, data-val-regex y data-

val-range). Como se puede ver, la URL en la barra de direcciones sigue siendo /Producto/Create y no se ha hecho ninguna solicitud al servidor; la validación se lleva a cabo totalmente en el navegador, antes de enviar cualquier información. Si bien esto optimiza la experiencia del usuario al proporcionar retroalimentación instantánea, no basta por sí mismo, pues el usuario puede desactivarlo o evitarlo (ver siguiente captura).

Validación en el servidor (con JavaScript deshabilitado)



Se desactivó JavaScript en el navegador para esta captura (F12 → Ctrl+Shift+P → "Disable JavaScript"), lo que resultó en la eliminación de toda la capa de validación del cliente. Cuando el formulario se envía con datos vacíos o inválidos, el navegador no puede validar ni interceptar nada; la solicitud POST se remite directamente al servidor con los datos exactamente como fueron introducidos. En el controlador ProductoController, el método Create procesa la solicitud y examina ModelState.IsValid. Este último verifica las mismas reglas que fueron establecidas en el modelo Producto a través de Data Annotations ([RegularExpression], [Required], [Range], etc.), pero ahora se ejecutan en C# del lado del servidor. Ya que el modelo no es válido, la acción ejecuta return View(producto), lo que devuelve la misma vista con los mensajes de error adecuados, tal como sucede cuando JavaScript está activado. Esta captura demuestra que la integridad de los datos no está sujeta al navegador: aun cuando un usuario con malas intenciones desactive JavaScript o modifique el formulario directamente (por medio de instrumentos como Postman, por ejemplo), el servidor continúa rechazando los datos inválidos.

4. Publicación en la rama feature/practica-aspnet

Los cambios se confirmaron mediante Git y se publicaron en una rama dedicada, separada de main, siguiendo el flujo estándar de trabajo por funcionalidades (feature branch).

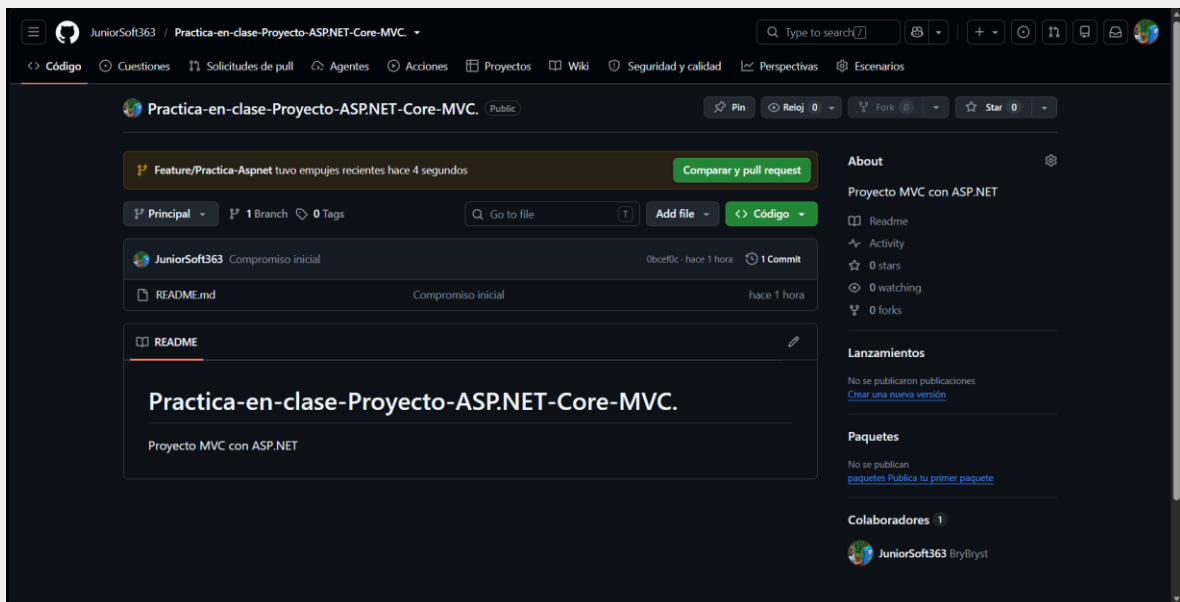
Comandos utilizados

```
git remote add origin https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC..git
git remote -v
git push -u origin feature/practica-aspnet
```

```

PS C:\MiPrimerMVC> git remote add origin https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC..git
PS C:\MiPrimerMVC> git remote -v
origin  https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC..git (fetch)
origin  https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC..git (push)
PS C:\MiPrimerMVC> git push -u origin feature/practica-aspnet
Enumerating objects: 27, done.
Counting objects: 100% (27/27), done.
Delta compression using up to 8 threads
Compressing objects: 100% (24/24), done.
Writing objects: 100% (27/27), 9.38 KiB | 384.00 KiB/s, done.
Total 27 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
remote:
remote: Create a pull request for 'feature/practica-aspnet' on GitHub by visiting:
remote:   https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC./pull/new/feature/practica-aspnet
remote:
To https://github.com/JuniorSoft363/Practica-en-clase-Proyecto-ASP.NET-Core-MVC..git
 * [new branch]      feature/practica-aspnet -> feature/practica-aspnet
branch 'feature/practica-aspnet' set up to track 'origin/feature/practica-aspnet'.

```



5. README con instrucciones de ejecución

Se documentó el proyecto en un archivo README.md ubicado en la raíz, con los requisitos previos, los pasos de instalación y ejecución, la tabla de rutas disponibles, y una breve explicación de la capa de validación y protección CSRF.

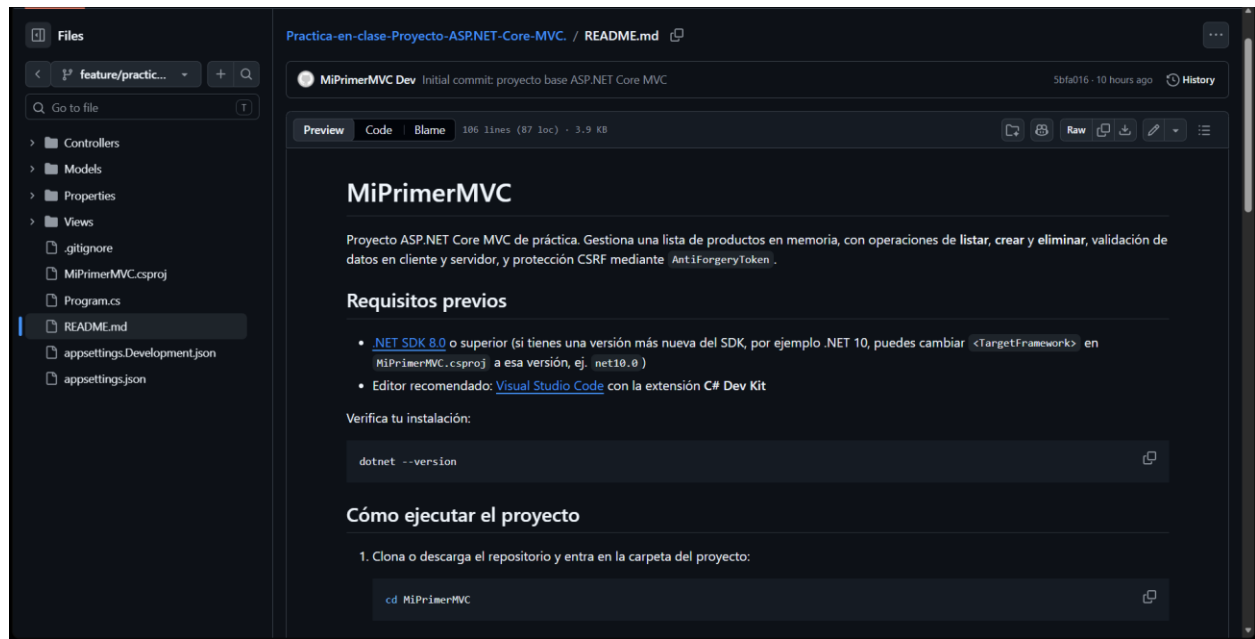
Extracto README.md

```

## Cómo ejecutar el proyecto

1. cd MiPrimerMVC
2. dotnet restore
3. dotnet run
4. Abrir http://localhost:5080 en el navegador

```

Conclusión

El proyecto cumple con los cinco requisitos solicitados: listado de productos en tabla HTML, eliminación segura con confirmación previa y protección contra CSRF, doble capa de validación (cliente y servidor), control de versiones mediante una rama de funcionalidad dedicada, y documentación de ejecución a través de un README. El código se encuentra disponible en el repositorio de GitHub, en la rama feature/practica-aspnet.